

Implementation Status Report

Matrimony API — features delivered and changes made

Report date: 4 October 2026 · Repository: D:\matrimony · Branch: master · Phase 0-3 complete

23 API ENDPOINTS	31 DATABASE MODELS	390 AUTOMATED TESTS	19 TEST FILES	8 COMMITTS
---------------------	-----------------------	------------------------	------------------	---------------

1. Executive summary

The API is functionally complete through **Phase 3 (Discovery)**. A user can register and sign in, hold a profile with photos and master-data attributes, and browse other users through a two-way net-worth visibility preference instead of the earlier same-category partition. Every phase was verified against a real PostgreSQL 16 instance, not only against unit-test doubles.

The most recent delivery replaced hard same-category discovery with two independently editable preferences. This is the single largest behavioural change in the project so far, and it changed a pricing rule: with cross-band discovery possible, the unlock price must come from the **target's** category rather than the viewer's, so the amount is knowable before checkout.

Three defects that were invisible to unit tests surfaced during live verification and are now fixed and covered — details in section 8. All 390 tests pass and the full `npm run verify` pipeline (Prisma validation and generation, typecheck, lint, test, build) exits clean.

2. Phase status

Phase	Name	Status	Delivered behaviour
0	Foundations	Complete	Monorepo, shared contracts, 31-table schema, config validation, security headers, rate limiting, clock service with drift guard, health probes, CI that boots the API against real PostgreSQL 16.
1	Identity and registration	Complete	OTP request and verify, registration completing server-side category assignment, access JWT (15 min) with rotating opaque refresh tokens, login/logout/refresh/ <code>GET /auth/me</code> , account logout.
2	Profile and master data	Complete	Profile CRUD with per-section validation, profile completeness, visibility toggle, master-list endpoints, direct-to-storage photo upload with presigned URLs. Photos are never paywalled; only <code>APPROVED</code> photos reach another user.
3	Discovery	Complete	Two-way visibility rule, eligibility gates, saved preferences, temporary narrowing filters, cursor pagination, locked preview cards priced from the target's category.

4	Payments and unlock delivery	Next	Razorpay orders with snapshotted pricing, webhook idempotency, ContactUnlock only from a settled payment, 24-hour expiry plus the 2-hour reminder. Blocked on GAP-3 (sort precedence).
5	Trust and safety	Planned	Reports, blocking, moderation queue, suspension and deletion with the retention-then-anonymise rule.
6	Identity verification	Planned	Cashfree Secure ID via DigiLocker; only provider reference and last four digits persisted. No user is discoverable until this passes.
7	Notifications	Planned	Push (FCM HTTP v1), SMS (DLT-gated), email, in-app, including the unlock-expiry reminder.
8	Admin API and dashboard	Planned	RBAC by permission, audit logging on every mutating admin action, master-data and pricing management, Next.js dashboard.
9	Mobile app	Planned	React Native client against the contracts in <code>packages/shared</code> .

3. API endpoint inventory

All 23 implemented endpoints, grouped by controller. Paths are relative to the `/api` prefix.

Method	Path	Purpose
Health		
GET	<code>/health/live</code>	Liveness probe.
GET	<code>/health/ready</code>	Readiness probe, including the database.
Identity		
POST	<code>/auth/otp/request</code>	Issue an OTP. Codes stored hashed; cooldown and attempt limits enforced server-side.
POST	<code>/auth/otp/verify</code>	Verify and consume the OTP atomically; report whether registration is still required.
POST	<code>/auth/register</code>	Complete registration and assign the server-controlled net-worth category.
POST	<code>/auth/refresh</code>	Rotate the refresh token. Reuse of a rotated token revokes the whole family.
POST	<code>/auth/logout</code>	Revoke the session.
GET	<code>/auth/me</code>	Current user, including the immutable own category.
Profile		
POST	<code>/profile</code>	Create the profile for the authenticated user.
GET	<code>/profile</code>	Own profile.
PATCH	<code>/profile</code>	Update profile fields with per-section validation.
GET	<code>/profile/completeness</code>	Completeness scoring.
PATCH	<code>/profile/visibility</code>	Show or hide the profile from discovery.
POST	<code>/profile/photos/upload</code>	Presigned direct-to-storage upload.
POST	<code>/profile/photos/complete</code>	Confirm the uploaded object and queue it for approval.
GET	<code>/profile/photos</code>	Own photo list.
PATCH	<code>/profile/photos/:photoId/primary</code>	Set the cover photo.
DELETE	<code>/profile/photos/:photoId</code>	Remove a photo.
Master data		
GET	<code>/master-data/lists</code>	Community, education and profession values.

GET	/master-data/net-worth-categories	NEW The four active bands with bounds as strings and the setup-fee quote.
Visibility and discovery		
GET	/me/visibility	NEW Stored lists, the configured flags, and the immutable own category.
PATCH	/me/visibility	NEW Save one or both directions; partial replacement semantics.
GET	/discovery	NEW Locked candidate cards, scope actually searched, cursor pagination.

4. Features implemented in this cycle

4.1 Two-way net-worth visibility and discoverability

The prior rule was that a user only ever saw profiles in their own net-worth band. That is replaced by two independent, user-editable lists:

- **discover** — the bands this user wants to browse.
- **visible_to** — the bands allowed to browse this user.

A candidate is returned only when **both** hold:

1. the candidate's band appears in the viewer's effective **discover** list, and
2. the viewer's band appears in the candidate's effective **visible_to** list.

Cross-band discovery is therefore opt-in on both sides and cannot happen by accident. Verified over HTTP: withdrawing either side's opt-in immediately removes the profile from the feed, and a band that nobody accepts is not even reported as searched.

4.2 Category options endpoint

GET /master-data/net-worth-categories serves the four active bands in display order with their bounds as strings, because the values exceed the safe integer range in JavaScript. **PENDING_REVIEW** is returned but flagged **is_discoverable: false**, which is a property of the band rather than a per-user permission. Each band carries its setup-fee quote.

Setup fee confirmed: ₹15.00 base plus 18% GST = ₹17.70 payable.

GAP-5 resolved 2026-10-03. GST is added on top, not included. One per account lifetime, collected only after ID verification, recorded under a distinct payment purpose so it can never be confused with a contact unlock.

4.3 Locked discovery cards

- Exactly four keys per card: **profile_id**, **photos**, **profile_locked**, **unlock_price**.
- Built by explicit projection, never an object spread, so the hidden-field rule is enforced by the shape of the response rather than by code review.
- No name, contact, income, religion, community, horoscope or object key is exposed. Asserted by scanning the whole serialized payload, not just the documented fields.

- Unlock price derived from the **target's** category, so it is known before checkout. Verified: a cross-band card quoted $499.00 + \text{GST} = 588.82$ while the same feed's own-band card quoted $249.00 + \text{GST} = 293.82$.
- Only `APPROVED` photos are exposed, through per-request presigned URLs.

4.4 Eligibility and safety gates applied to the read path

A candidate must be an approved, active, non-deleted profile belonging to a user who is active, ID-verified, setup-fee paid, not anonymised, and not blocked with the viewer in either direction. Both block directions are expressed as `none` relations rather than joins, so a duplicate row cannot resurface a blocked user through the join.

5. Data model changes

31 models across 4 migrations. This cycle added the two-way preference model:

Change	Detail
<code>PartnerPreference.discoveryConfigured</code>	Boolean flag distinguishing "never saved" from "deliberately saved as nobody".
<code>PartnerPreference.visibilityConfigured</code>	Same distinction for the other direction.
<code>UserDiscoveryCategory</code>	Join table, composite key <code>(userId, category)</code> , so duplicates are impossible at the storage layer.
<code>UserVisibilityCategory</code>	Join table with the same composite key and reverse category indexes.

`User.networthCategory` remains server-controlled and cannot be changed by the user through any endpoint. That rule survived the specification change even though the surrounding discovery rule did not.

6. Business rules settled at implementation time

The specification was silent on three points. Each was decided explicitly and recorded in `docs/decisions/visibility-discoverability-2026-10-03.md` rather than inferred at runtime.

Question	Decision and rationale
What does an unset list mean?	The user's own band only. Behaviour is identical to the old partition until the user opts in, so nobody is silently exposed to a band they never chose.
What does an empty list mean?	Nobody, validly. It doubles as a pause control, which is why explicit <code>configured</code> flags are required rather than inferring intent from an empty table.
Can a request widen scope?	No. A per-request category is rejected by the contract. This is why <code>GET /me/visibility</code> returns raw stored lists plus flags instead of fabricating the default into the response.

7. Defects found and fixed

Each of the three was unreachable from unit tests and was caught by running against real PostgreSQL. All three now have regression coverage.

7.1 Foreign `Decimal` instances threw on every priced response

FIXED `toDecimal` tested `instanceof Decimal`. Prisma's driver adapter constructs Decimals from its own bundled copy of `decimal.js`, so the check fails for any value read from the database. Such a value fell through to `value.trim()` and threw `value.trim is not a function`, producing a 500 on every endpoint that returns money. Now duck-typed, with `toFixed` rather than `toString` so a copy configured for exponent notation still parses. The saved preference body deliberately keeps strict types, because there a wrongly typed value is a client bug worth surfacing.

7.2 Every paginated request was rejected

FIXED Query parameters arrive as strings, so `?limit=1` failed a plain `z.number()` with "Expected number, received string". Pagination was unusable over HTTP while passing in unit tests, which construct typed objects directly. Numeric and boolean query fields now coerce, with one deliberate exception: `use_saved_preference` does **not** use `z.coerce.boolean()`, because that maps every non-empty string to true and would invert an explicit `false`.

7.3 The response claimed a preference had been ignored when it had not

FIXED `preference_applied` echoed the request flag, while the scope was in fact always computed from the saved preference. Sending `use_saved_preference=false` produced a response claiming preferences were skipped on cards that preferences had filtered. Now always `true`, with the flag documented as compatibility-only, since honouring it would reintroduce the scope-widening bypass.

7.4 Two false alarms worth recording

Both looked like product bugs and were faults in the test fixtures:

- **Cursor pagination appeared broken.** The seeded profile IDs were 19 characters while the cursor validator requires 20–32, as real cuids are 25. Rejected IDs silently reset pagination to page 1. The validator was correct; the fixture was unrealistic. Fixtures now generate cuid-shaped IDs.
- **The never-saved default appeared missing.** `GET /me/visibility` correctly returns empty lists for an unsaved account, with the flags false and the own category reported alongside. The own-band default is applied when the preference is resolved for a search, not written back as though the user had chosen it.

8. How it was verified

Two independent strategies, because each catches what the other misses.

8.1 Automated suite

Gate	Result	Notes
<code>prisma validate and generate</code>	Pass	Client 7.10.0 generated into the API source tree.
Typecheck (shared + API)	Pass	Strict TypeScript across both workspaces.
Lint (shared + API)	Pass	ESLint, no suppressions added.
Shared tests	148 passed	6 suites: contracts, money, enums, profile.
API tests	242 passed	13 suites including the app-boot check against a real module graph.
Build (shared + API)	Pass	<code>tsc --build</code> and <code>nest build</code> .
<code>npm run verify</code>	Exit 0	Whole pipeline green, the gate required before any commit.

8.2 Live HTTP and database verification

Two scripts exercised the running API against real PostgreSQL 16, through the full OTP authentication dance and real database rows rather than doubles. **88 assertions, all passing.**

Area	Assertions	What it proved
Categories, preferences, auth	35	Band list and setup-fee totals; partial PATCH; unknown band rejected with the offending key named; the review bucket refused; retired band refused; a refused save writes nothing; own category immutable; unsaved preference reports raw state; empty list pauses the feed; authentication still required on every route.
Two-way discovery	53	Empty scope when nobody accepts; target opting the viewer in; cross-band discovery working both ways; target-band pricing; withdrawal on either side; no locked field anywhere in the payload; exactly the four contracted keys; per-request category rejected; cursor paging with no overlap; malformed cursor restarts rather than erroring; unauthenticated access refused.

Eligibility could not be reached through the API alone, because the ID check and the payment webhook that grant verified status and paid status do not exist yet. Those accounts were promoted directly in SQL to make the feed testable end to end, with that shortcut recorded in the fixture itself.

9. Known limitations and open work

9.1 Deliberately deferred in delivered phases

Gap	Effect
-----	--------

ProfileAttribute CRUD absent	Admin-configurable profile attribute values exist in the schema but have no management endpoints yet.
Explicit null clearing	Clearing education_id , profession_id or community_id is skipped rather than unsetting the column.
Photo editability and TTL	Photo mutations do not consistently re-check account and profile editability, and initiation hardcodes a 300-second presign TTL instead of using S3_SIGNED_URL_TTL .

9.2 Open decisions required from the client

Gap	Subject	Status
GAP-3	Discovery sort precedence	Blocks Phase 4. "Newest and closest" is two orderings; the primary sort changes the index design and the query. Currently newest-first. This is the one input needed to start payments.
GAP-1	Unlocks bought from a profile that is then deleted	Client's answer means the buyer loses access with no refund. A credit note is recommended.
GAP-2	Boundary values between bands	Data-integrity question, needed before the category-selection screen ships.
GAP-4	Family photos	Required or optional, which is the cover image, and whether it is visible before payment. Face detection in group photos is materially harder.
GAP-6	Who may delete a user	Super Admin recommended.
GAP-7	Legal documents are a new line item	Privacy Policy and Terms of Service should be reflected in contract and quote.
GAP-8	UI/UX ownership moved in-house	18 user-facing and 14 admin screens need confirming against the estimate.
GAP-9	Aadhaar handling	Implemented as reference plus masked last four only; needs client comfort that the full number is never held.
GAP-10	Do preferences filter at all?	A yes/no is needed; filtering could make some profiles permanently undiscoverable.
GAP-11	Cloudflare R2 data residency	Not clean: R2 cannot guarantee India-only residency. Blocked externally.
GAP-12	SMS and email providers	Production provider and TRAI DLT registration outstanding; only the non-production dev_code path works.

9.3 Environment and tooling limits

- **Live object storage untested.** The configured MinIO image is unavailable, so a real PUT has not been exercised. Presigned URL generation itself is verified.
- **Missing provider credentials:** Razorpay, Cashfree, Firebase, domain, SMS/email, DLT. All external; none block Phases 1–3.
- **API test suite memory ceiling.** With roughly 2.6 GB free RAM the API suite intermittently exhausts the Node heap when Jest runs its default worker count. It passes reliably with --maxWorkers=2 . This is a constraint of the

development machine, not a code defect, so the repository default has not been changed — flagging it for a decision.

10. Commit history

Commit	Date	Description
44ee52a	2026-10-04	Keep the visibility specification; drop the HTML it was built from.
fd0051c	2026-10-04	Replace same-category discovery with two editable preferences. Adds the visibility and discovery endpoints, the category options endpoint, and the three defect fixes.
ed824e1	2026-10-03	Close three identity holes: OTP replay on an already-registered number, lockout never called from the request path, and refresh ignoring suspension.
5682bab	2026-10-02	Add profiles, master data and direct-to-storage photo upload.
acff6e3	2026-10-02	Let users register, sign in, and rotate their sessions.
03ed948	2026-10-01	Record the client's supplier decisions, and the one that does not work.
8a358fd	2026-10-01	Make the API actually boot, and verify it against a real database.
6f31a64	2026-09-29	Foundation: monorepo, schema and the client answers of 2026-09-29.

Working tree is clean at the time of writing. The visible HTML file that was generated into the client PDF has been removed as a redundant intermediate; the PDF it produced remains the deliverable.

Implementation status report — Matrimony API — generated 4 October 2026 from commit 44ee52a on branch master. Figures are taken from the repository itself: 31 Prisma models, 4 migrations, 23 mapped endpoints, 390 automated tests across 19 files.